



THE UNIVERSITY OF  
MELBOURNE

FINAL REPORT

---

# The Art of Scientific Computing: Project Title: High Dimensional Kalman Filters

---

*Author:*  
Steven Pavli

*Subject Handbook code:*  
COMP-90072

Faculty of Science  
The University of Melbourne

Original Version: 9th October 2026

Subject co-ordinators: A/Prof. Suzie Sheehy & Prof. Christian Reichardt

# Contents

|   |                               |          |
|---|-------------------------------|----------|
| 1 | Background and Goal . . . . . | 1        |
| 2 | Derivations . . . . .         | 1        |
| 3 | Dynamic Projections . . . . . | 2        |
| 4 | Dynamic PCA-KF . . . . .      | 2        |
| 5 | Results . . . . .             | 3        |
| 6 | Discussion . . . . .          | 4        |
|   | <b>Bibliography</b>           | <b>4</b> |

## 1 Background and Goal

The theme of this project is investigating the use of the Kalman Filter on high-dimensional time series data. We can specify the model as follows:

$$\beta_t = \beta_{t-1} + \epsilon_t \quad (1)$$

$$y_t = \beta_t^T \mathbf{x}_t + z_t \quad (2)$$

where  $\mathbf{x}_t, \beta_t \in \mathbb{R}^p$ ,  $p \geq 1$  large, and  $\epsilon_t \sim N(0, \Sigma_t)$ ,  $z_t \sim N(0, \tau_t^2)$ . Note that  $z_t$  is independent of  $\mathbf{x}_t$ . Equations (1) and (2) are the state and observation process equations respectively. In this case, we take  $(\mathbf{x}_t, y_t)_{1 \leq t \leq T}$  be observed, while  $\beta_t$ ,  $\Sigma_t$ ,  $\tau_t$  are unobserved (latent).

The goal is to efficiently estimate the  $(\beta_t)$  process (the loadings) via a Kalman Filter. To this end, we will consider a dimensionality reduction approach by applying PCA to  $\mathbf{x}_t$ , and running the Kalman Filter in this space, then back-transform our loading estimates at the end.

We will make use of Python, and NumPy in particular for fast matrix operations.

## 2 Derivations

Notation:

- $\beta_t$ : the state variable
- $Q_t = \Sigma_t$  is the covariance of the state process noise.
- $P_{t|t}$  is the covariance of our estimate of  $\beta_t$ .
- $R_t = \tau_t^2$  is the observation noise.

Prediction:

$$\begin{aligned} \beta_{t|t-1} &= \beta_{t-1|t-1} \\ P_{t|t-1} &= P_{t-1|t-1} + \Sigma_t \end{aligned}$$

Update:

$$\begin{aligned} \tilde{y}_t &= y_t - \beta_{t|t-1}^T \mathbf{x}_t \\ S_t &= \mathbf{x}_t^T \Sigma_t \mathbf{x}_t + \tau_t^2 \sim \text{Cov}(\tilde{y}_t) \\ K_t &= \dots \end{aligned}$$

### 2.1 Initialisation

TO ADD

## 2.2 PCA-KF

The idea is that if  $p$  is large, then we apply dimension reduction to  $\mathbf{x}_t$ . This assumes the data generating process doesn't change 'too much' in the training set.

Observe that if  $UU^T \approx I_p$ , then

$$\mathbf{y}_t \approx \beta_t^T UU^T \mathbf{x}_t + \mathbf{z}_t$$

This is the case, for example, if  $U$  represent PCA loadings with sufficient proportion of variance retained.

## 3 Dynamic Projections

Suppose we wish to update our PCA loadings as we progress through the iterations. This is a reasonable feature since the loadings estimates may become stale if we keep them static after initialisation. However, one issue arises from this; if we change our projection, then the usual update equations do not adapt to this. For example, the update  $P_{k+1} = P_k + Q_k$  assumes the representation given by the  $k$ -th projection, likewise for updating the state estimate.

To this end, we can consider a projection-chaining approach which keeps the dynamics of the updating equation identical, but for a projection-approximation step at the start of update. Here, we will keep the loadings minimally restricted:  $U \in \mathbb{R}^{q \times p}$  with  $q < p$  and  $\text{rank}(U_k) = q$ . That is, we do not require orthonormality of columns, nor do these need to represent PCA loadings.

If we have the projected data:  $\mathbf{z}_t = \mathbf{x}_t U_t$ , then we typically reverse this transformation with  $\mathbf{x}_t \approx \mathbf{z}_t U_t^+$  where  $U_t^+ = U_t^T (U_t^T U_t)^{-1}$  is the pseudoinverse of  $U_t$ . To push  $\mathbf{z}_t$  to a new basis,  $U_{t+1}$ , then we can do this approximately with:

$$\begin{aligned} \mathbf{z}'_t &= \mathbf{x}_t U_{t+1} \\ &\approx \mathbf{x}_t U_t U_t^+ U_{t+1} \\ &= \mathbf{z}_t U_t^+ U_{t+1} \end{aligned}$$

We note that every time we apply an approximate projection, we permanently lose information in the direction of the bases lost in the prior projection. Note that we cannot simply use the projection  $U_{t+1}$  on its own since the underlying state  $\mathbf{x}_t$  is unknown. Hence, this chaining step is necessary to keep the states in the correct basis.

### 3.1 Rotating Covariance

We propose a modification to the usual structure by allowing the PCA-loadings of the  $\mathbf{x}$  data to rotate over time, while keeping the component variances (eigenvalues) constant. We can do this by using a block-rotation matrix  $R$  with reasonably small angles, and then we create the new covariance matrix via,

$$\Sigma' = R \Sigma R^T$$

Observe that implicitly we are rotating the PCA loadings since  $R \Sigma R^T = R Q \Lambda Q^T R^T = Q' \Lambda (Q')^T$ , where  $Q' = RQ$ . To be more precise,  $R$  is constructed as  $R = \text{diag}(R(\theta_1), \dots, R(\theta_k))$  for  $k = p/2$  if  $p$  is even, else  $R = \text{diag}(1, R(\theta_1), \dots, R(\theta_{\lfloor p/2 \rfloor}))$ ; see [1]. Hence, to rotate the data incrementally, we can start with some covariance matrix  $\Sigma_0$ , then define:

$$\Sigma_n = R^n \Sigma_0 (R^n)^T$$

There is a question of how to best optimise this for a large dataset. One can directly construct  $R^n$  for each  $n$  by replacing  $R(\theta_j)$  with  $R(n\theta_j)$  for each  $j = 1, \dots, k$ , hence eliminating the need to compute the matrix power. From there, we can batch multiply to calculate the entire sequence  $(\Sigma_1, \dots, \Sigma_N)$  simultaneously, provided memory constraints are respected.

### 3.2 Evaluating Approximation Error

Due to information loss which results from the projection chaining, we can test how much information is preserved as compared to simply projecting once initially. This can give us a sense of how we should best project our data during the Kalman filter steps.

## 4 Dynamic PCA-KF

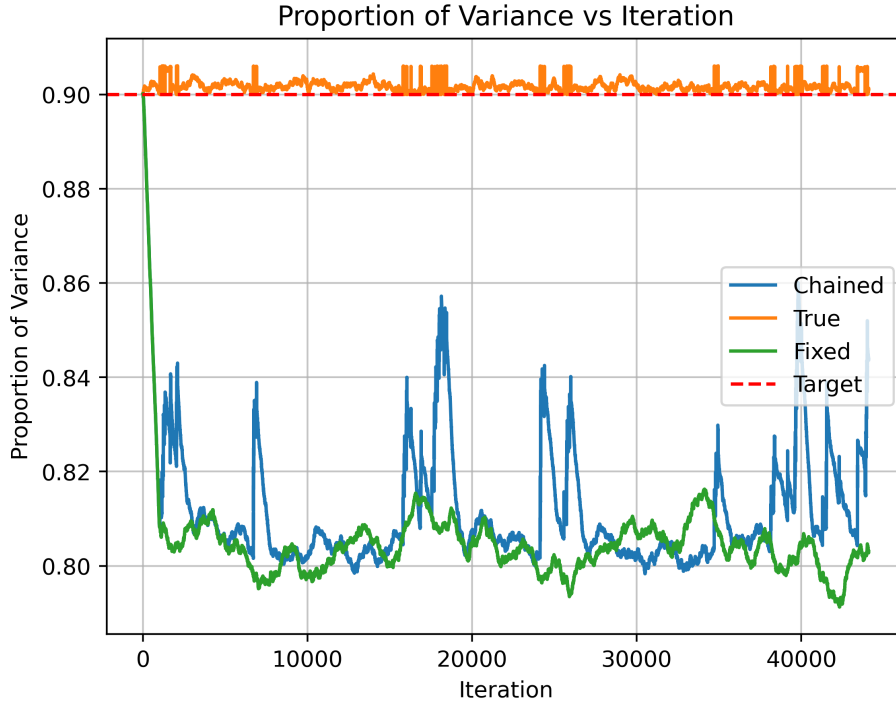
This applies the previous projection chaining approach to the PCA-KF algorithm.

## 5 Results

### 5.1 Dynamic Projection Tests

First we present a simulation result with the following hyperparameters:

- $(n, n_{burn}, p) = (50000, 1000, 100)$
- Angle increment (in radians)  $\theta = 0.05$ , or,  $\frac{2\pi}{\theta} \approx 126$
- $\text{Var}(z_t) = 10^{-2}$
- $\varepsilon_t$  construction scale parameter is  $10^{-2}$
- $x_t$  construction scale parameter is  $10^{-1}$



**Figure 1:** Proportion of variance retained by method.

We observe from Figure 1 that the Chained approach is generally more performant than with the fixed projection, however, it is not always better. Note that our projection chaining does not assume anything about the rotation structure; it just cares that the loadings are changing over time. One can entertain the idea that if a rotation structure is assumed, then all that is needed is to estimate the angle step in each plane, then in theory we circumvent the information loss issue. However, the rotation structure does not have an immediate real world application, so there is no grounds to even assume this when chaining projections.

### 5.2 Static Kalman Filter

Simulations have been largely done, but still need to produce the plots. Will provide:

- Residual accuracy (based on a normalised version of  $\text{var}(\tilde{y})$ ) using the estimated  $\beta$ , maximised over initial choices of  $P, Q \in \{10^{-4}I, 10^{-3}I, 10^{-2}I, 10^{-1}I\}$ .
- Compare a few dimensions of  $\beta$  vs.  $\hat{\beta}$ .
- Time performance.

### 5.3 Dynamic Kalman Filter

Have only run small scale tests so far. Will produce same plots/tables as above.

## 6 Discussion

TO DO.

# Bibliography

- [1] rschwieb (<https://math.stackexchange.com/users/29335/rschwieb>). Define rotation matrices for arbitrary dimension? Mathematics Stack Exchange. URL:<https://math.stackexchange.com/q/386513> (version: 2013-05-10).